# STAT508 - HW2

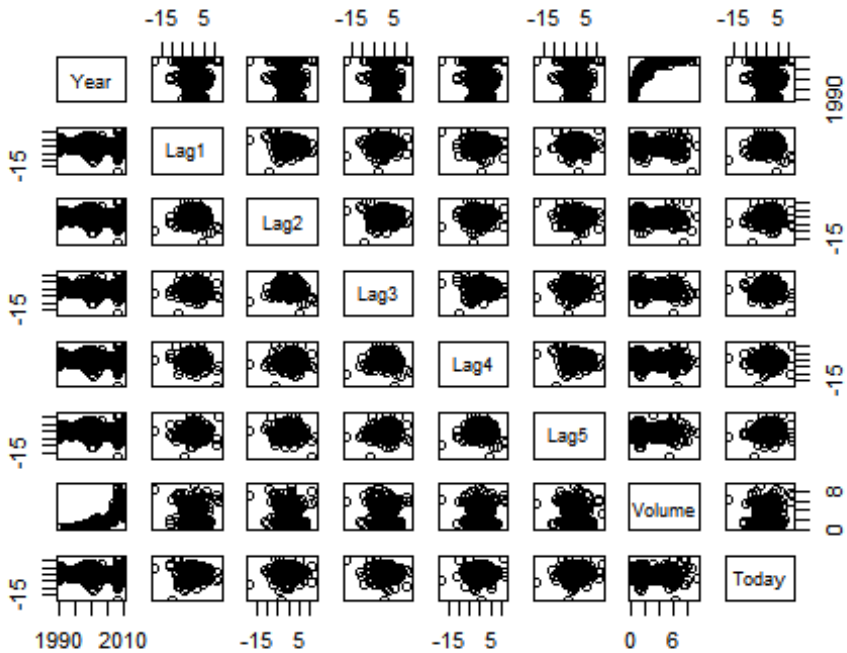
Ana Camargo

2023-03-16

## Chapter 4 - Problem 13

a) Produce some numerical and graphical summaries of the Weekly data. Do there appear to be any patterns?

```
library(ISLR2)
data("Weekly")

names(Weekly)

## [1] "Year"      "Lag1"      "Lag2"      "Lag3"      "Lag4"      "Lag5"
## [7] "Volume"    "Today"     "Direction"

dim(Weekly)

## [1] 1089    9

summary(Weekly)

##       Year           Lag1               Lag2               Lag3
##  Min.   :1990   Min.   :-18.1950   Min.   :-18.1950   Min.   :-18.1950
##  1st Qu.:1995   1st Qu.: -1.1540   1st Qu.: -1.1540   1st Qu.: -1.1580
##  Median :2000   Median :  0.2410   Median :  0.2410   Median :  0.2410
##  Mean   :2000   Mean   :  0.1506   Mean   :  0.1511   Mean   :  0.1472
##  3rd Qu.:2005   3rd Qu.:  1.4050   3rd Qu.:  1.4090   3rd Qu.:  1.4090
##  Max.   :2010   Max.   : 12.0260   Max.   : 12.0260   Max.   : 12.0260
##       Lag4               Lag5              Volume            Today
##  Min.   :-18.1950   Min.   :-18.1950   Min.   :0.08747   Min.   :-18.1950
##  1st Qu.: -1.1580   1st Qu.: -1.1660   1st Qu.:0.33202   1st Qu.: -1.1540
##  Median :  0.2380   Median :  0.2340   Median :1.00268   Median :  0.2410
##  Mean   :  0.1458   Mean   :  0.1399   Mean   :1.57462   Mean   :  0.1499
##  3rd Qu.:  1.4090   3rd Qu.:  1.4050   3rd Qu.:2.05373   3rd Qu.:  1.4050
##  Max.   : 12.0260   Max.   : 12.0260   Max.   :9.32821   Max.   : 12.0260
##  Direction
##  Down:484
##  Up  :605
##
##
##
##

pairs(Weekly[,-9])
```




```
cor(Weekly[,-9])

##               Year         Lag1        Lag2        Lag3         Lag4
## Year    1.00000000 -0.032289274 -0.03339001 -0.03000649 -0.031127923
## Lag1   -0.03228927  1.000000000 -0.07485305  0.05863568 -0.071273876
## Lag2   -0.03339001 -0.074853051  1.00000000 -0.07572091  0.058381535
## Lag3   -0.03000649  0.058635682 -0.07572091  1.00000000 -0.075395865
## Lag4   -0.03112792 -0.071273876  0.05838153 -0.07539587  1.000000000
## Lag5   -0.03051910 -0.008183096 -0.07249948  0.06065717 -0.075675027
## Volume  0.84194162 -0.064951313 -0.08551314 -0.06928771 -0.061074617
## Today  -0.03245989 -0.075031842  0.05916672 -0.07124364 -0.007825873
##                Lag5      Volume        Today
## Year   -0.030519101  0.84194162 -0.032459894
## Lag1   -0.008183096 -0.06495131 -0.075031842
## Lag2   -0.072499482 -0.08551314  0.059166717
## Lag3    0.060657175 -0.06928771 -0.071243639
## Lag4   -0.075675027 -0.06107462 -0.007825873
## Lag5    1.000000000 -0.05851741  0.011012698
## Volume -0.058517414  1.00000000 -0.033077783
## Today   0.011012698 -0.03307778  1.000000000
```

In the correlation plot it can be seen that there are some patterns in the data. Only Year vs Volume variables are highly correlated (0.84 correlation) which suggests that we could decide not to include both together when fitting a regression model.

b) Use the full data set to perform a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Use the summary function to

print the results. Do any of the predictors appear to be statistically significant? If so, which ones?

```
glm.fit <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
data=Weekly , family=binomial)
summary(glm.fit)

##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = binomial, data = Weekly)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -1.6949  -1.2565   0.9913   1.0849   1.4579
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.26686    0.08593   3.106   0.0019 **
## Lag1        -0.04127    0.02641  -1.563   0.1181
## Lag2         0.05844    0.02686   2.175   0.0296 *
## Lag3        -0.01606    0.02666  -0.602   0.5469
## Lag4        -0.02779    0.02646  -1.050   0.2937
## Lag5        -0.01447    0.02638  -0.549   0.5833
## Volume      -0.02274    0.03690  -0.616   0.5377
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1496.2  on 1088  degrees of freedom
## Residual deviance: 1486.4  on 1082  degrees of freedom
## AIC: 1500.4
##
## Number of Fisher Scoring iterations: 4
```

From the fitted logistic regression, there is only one predictor that results significant (at a 5% significance level), which is Lag2 (p-value = 0.0296). All the other predictors are not significant for this logistic regression model.

c) Compute the confusion matrix and overall fraction of correct predictions. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

```
glm.probs <- predict(glm.fit , type = "response")
glm.pred <- rep("Down", dim(Weekly)[1])
glm.pred[glm.probs > .5] = "Up"
table(glm.pred , Weekly$Direction)

##
## glm.pred Down  Up
```

```
##     Down   54  48
##     Up    430 557
```

```
mean(glm.pred == Weekly$Direction)
```

```
## [1] 0.5610652
```

The proportion of correct predictions obtained from the confusion matrix shows that 0.56 of the predictions are correctly classified. This means that 0.44 of the data is misclassified using this model.

d) Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Compute the confusion matrix and the overall fraction of correct predictions for the held out data (that is, the data from 2009 and 2010).

```
train <- (Weekly$Year <= 2008)
Weekly_2008.test <- Weekly[!train,]
glm.fit_2008 <- glm(Direction ~ Lag2 , data=Weekly, family=binomial,
subset=train)
summary(glm.fit_2008)
```

```
##
## Call:
## glm(formula = Direction ~ Lag2, family = binomial, data = Weekly,
##     subset = train)
##
## Deviance Residuals:
##    Min      1Q  Median      3Q     Max
## -1.536  -1.264   1.021   1.091   1.368
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.20326    0.06428   3.162  0.00157 **
## Lag2         0.05810    0.02870   2.024  0.04298 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1350.5  on 983  degrees of freedom
## AIC: 1354.5
##
## Number of Fisher Scoring iterations: 4
```

```
glm.probs_2008 <- predict(glm.fit_2008 , Weekly_2008.test, type = "response")
glm.pred_2008 <- rep("Down", dim(Weekly_2008.test)[1])
glm.pred_2008[glm.probs_2008 > .5] = "Up"
table(glm.pred_2008 , Weekly_2008.test$Direction)
```

```
##
## glm.pred_2008 Down Up
##          Down    9  5
##          Up     34 56
```

```
mean(glm.pred_2008 == Weekly_2008.test$Direction)
```

```
## [1] 0.625
```

Using the training data (years 1990-2008) to fit the model we also get Lag2 to be significant predictor in the model. When making predictions with the testing subset, we achieve a higher accuracy than before. The proportion of correct predictions is now 0.625.

e) Repeat (d) using LDA.

```
lda.fit <- MASS::lda(Direction ~ Lag2 , data=Weekly, subset=train)
lda.fit
```

```
## Call:
## lda(Direction ~ Lag2, data = Weekly, subset = train)
##
## Prior probabilities of groups:
##      Down        Up
## 0.4477157 0.5522843
##
## Group means:
##             Lag2
## Down -0.03568254
## Up    0.26036581
##
## Coefficients of linear discriminants:
##            LD1
## Lag2 0.4414162
```

```
lda.pred <- predict(lda.fit , Weekly_2008.test)
table(lda.pred$class, Weekly_2008.test$Direction)
```

```
##
##        Down Up
##   Down    9  5
##   Up     34 56
```

```
mean(lda.pred$class == Weekly_2008.test$Direction)
```

```
## [1] 0.625
```

When using LDA to fit the model with the same training set, and then making predictions with the testing set, the proportion of correct predictions stays same as in the previous case. The overall fraction of correct predictions for the held out data is still 0.625.

f) Repeat (d) using QDA.

```
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:ISLR2':
##
##     Boston
```

```
qda.fit <- qda(Direction ~ Lag2 , data=Weekly, subset=train)
qda.fit
```

```
## Call:
## qda(Direction ~ Lag2, data = Weekly, subset = train)
##
## Prior probabilities of groups:
##      Down        Up
## 0.4477157 0.5522843
##
## Group means:
##             Lag2
## Down -0.03568254
## Up    0.26036581
```

```
qda.pred <- predict(qda.fit , Weekly_2008.test)
table(qda.pred$class, Weekly_2008.test$Direction)
```

```
##
##        Down Up
##   Down    0  0
##   Up     43 61
```

```
mean(qda.pred$class == Weekly_2008.test$Direction)
```

```
## [1] 0.5865385
```

When using QDA to fit the model with the same training set, and then making predictions with the testing set, the proportion of correct predictions is lower than the obtained using logistic regression and LDA. This proportion is of 0.59, so now 0.41 of our testing data is being misclassified.

g) Repeat (d) using KNN with K = 1.

```
library(class)
train.X <- Weekly$Lag2[train]
test.X <- Weekly$Lag2[!train]
train.Direction <- Weekly$Direction[train]
set.seed(1)
knn.pred <- knn(as.data.frame(train.X), as.data.frame(test.X),
train.Direction, k=1)
table(knn.pred, Weekly_2008.test$Direction)
```

```
##
## knn.pred Down Up
```

```
##     Down   21 30
##     Up     22 31

mean(knn.pred == Weekly_2008.test$Direction)

## [1] 0.5
```

When using KNN (k=1) to fit the model with the same training set, the proportion of correct predictions in the testing set is even lower than all previous methods. This proportion is of 0.5, so now only half of the testing data is correctly classified.

h) Repeat (d) using naive Bayes.

```
library(e1071)
nb.fit <- naiveBayes(Direction ~ Lag2 , data=Weekly, subset=train)
nb.fit

##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##      Down        Up
## 0.4477157 0.5522843
##
## Conditional probabilities:
##       Lag2
## Y             [,1]     [,2]
##   Down -0.03568254 2.199504
##   Up    0.26036581 2.317485

nb.pred <- predict(nb.fit , Weekly_2008.test)
table(nb.pred, Weekly_2008.test$Direction)

##
## nb.pred Down Up
##    Down    0  0
##    Up     43 61

mean(nb.pred == Weekly_2008.test$Direction)

## [1] 0.5865385
```

When using Naive Bayes classifier the proportion of correct predictions in the testing set is 0.59, so it perfoms better than QDA and KNN, but worse than logistic regression and LDA.

i) Which of these methods appears to provide the best results on this data? The highest proportion of correct predictions, obtained from the confusion matrix of

each model, is obtained when fitting the model using logistic regression and LDA (0.625).

j) Experiment with different combinations of predictors, including possible transformations and interactions, for each of the methods. Report the variables, method, and associated confusion matrix that appears to provide the best results on the held out data. Note that you should also experiment with values for K in the KNN classifier.

When experimenting with different combinations of predictors and interactions, it turned out that these were not significant to the fitted models. As we saw from the beginning, the predictor that was significant among all of the 6 in the data set is only "Lag2". So, based on this, this is why the models are not significantly influenced by the other combinations. However, the case for KNN is not the same as we could try different values of k using the same predictor "Lag2". The results are shown below.

```
## with k=10
set.seed(1)
knn.pred_exp <- knn(as.data.frame(train.X), as.data.frame(test.X),
train.Direction, k=10)
table(knn.pred_exp, Weekly_2008.test$Direction)

##
## knn.pred_exp Down Up
##         Down   17 21
##         Up     26 40

mean(knn.pred_exp == Weekly_2008.test$Direction)

## [1] 0.5480769

#with k=50
set.seed(1)
knn.pred_exp2 <- knn(as.data.frame(train.X), as.data.frame(test.X),
train.Direction, k=180)
table(knn.pred_exp2, Weekly_2008.test$Direction)

##
## knn.pred_exp2 Down Up
##          Down    9  4
##          Up     34 57

mean(knn.pred_exp2 == Weekly_2008.test$Direction)

## [1] 0.6346154
```

From above results, we can see that using k=180 we get a higher proportion of correct predictions than when using k=1 and k=10. So, using a higher value of k in this case improves the model accuracy to 63.5% in this case.

## Chapter 4 - Problem 14

a) Create a binary variable, mpg01, that contains a 1 if mpg contains a value above its median, and a 0 if mpg contains a value below its median. You can compute the median using the median() function. Note you may find it helpful to use the data.frame() function to create a single data set containing both mpg01 and the other Auto variables.

```
data("Auto")
mpg01 <- rep(1,dim(Auto)[1])
mpg01[Auto$mpg < median(Auto$mpg)] = 0
Auto_2 <- cbind.data.frame(Auto,mpg01)
```

b) Explore the data graphically in order to investigate the association between mpg01 and the other features. Which of the other features seem most likely to be useful in predicting mpg01? Scatter plots and box plots may be useful tools to answer this question. Describe your findings.

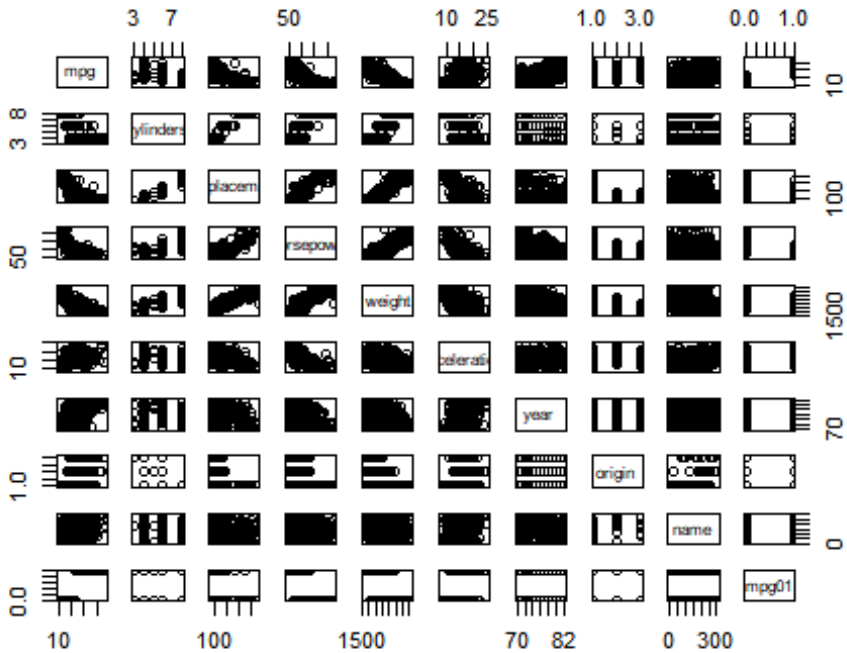
```
names(Auto_2)

##  [1] "mpg"          "cylinders"    "displacement" "horsepower"   "weight"
##  [6] "acceleration" "year"         "origin"       "name"         "mpg01"

dim(Auto_2)

## [1] 392  10

summary(Auto_2)

##       mpg          cylinders      displacement     horsepower
weight
##  Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.
:1613
##  1st Qu.:17.00   1st Qu.:4.000   1st Qu.:105.0   1st Qu.: 75.0   1st
Qu.:2225
##  Median :22.75   Median :4.000   Median :151.0   Median : 93.5   Median
:2804
##  Mean   :23.45   Mean   :5.472   Mean   :194.4   Mean   :104.5   Mean
:2978
##  3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:275.8   3rd Qu.:126.0   3rd
Qu.:3615
##  Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.
:5140
##
##   acceleration        year           origin                      name
##  Min.   : 8.00   Min.   :70.00   Min.   :1.000   amc matador       :  5
##  1st Qu.:13.78   1st Qu.:73.00   1st Qu.:1.000   ford pinto        :  5
##  Median :15.50   Median :76.00   Median :1.000   toyota corolla    :  5
##  Mean   :15.54   Mean   :75.98   Mean   :1.577   amc gremlin       :  4
##  3rd Qu.:17.02   3rd Qu.:79.00   3rd Qu.:2.000   amc hornet        :  4
##  Max.   :24.80   Max.   :82.00   Max.   :3.000   chevrolet chevette:  4
##                                                  (Other)           :365
##      mpg01
```

```
##  Min.   :0.0
##  1st Qu.:0.0
##  Median :0.5
##  Mean   :0.5
##  3rd Qu.:1.0
##  Max.   :1.0
##
```

```
cor(Auto_2[,-9])
```

```
##                     mpg  cylinders displacement horsepower     weight
## mpg           1.0000000 -0.7776175   -0.8051269 -0.7784268 -0.8322442
## cylinders    -0.7776175  1.0000000    0.9508233  0.8429834  0.8975273
## displacement -0.8051269  0.9508233    1.0000000  0.8972570  0.9329944
## horsepower   -0.7784268  0.8429834    0.8972570  1.0000000  0.8645377
## weight       -0.8322442  0.8975273    0.9329944  0.8645377  1.0000000
## acceleration  0.4233285 -0.5046834   -0.5438005 -0.6891955 -0.4168392
## year          0.5805410 -0.3456474   -0.3698552 -0.4163615 -0.3091199
## origin        0.5652088 -0.5689316   -0.6145351 -0.4551715 -0.5850054
## mpg01         0.8369392 -0.7591939   -0.7534766 -0.6670526 -0.7577566
##              acceleration       year     origin      mpg01
## mpg             0.4233285  0.5805410  0.5652088  0.8369392
## cylinders      -0.5046834 -0.3456474 -0.5689316 -0.7591939
## displacement   -0.5438005 -0.3698552 -0.6145351 -0.7534766
## horsepower     -0.6891955 -0.4163615 -0.4551715 -0.6670526
## weight         -0.4168392 -0.3091199 -0.5850054 -0.7577566
## acceleration    1.0000000  0.2903161  0.2127458  0.3468215
## year            0.2903161  1.0000000  0.1815277  0.4299042
## origin          0.2127458  0.1815277  1.0000000  0.5136984
## mpg01           0.3468215  0.4299042  0.5136984  1.0000000
```

```
pairs(Auto_2)
```




The variable mpg01 seems to be associated with some of the variables in the dataset, those we could set as predictors for the models we will be fitting.the features that seem most likely to be useful in predicting mpg01 are: displacement, horsepower, weight. The "mpg" variable is not included as this was used to create mpg01.

c) Split the data into a training set and a test set.

```
set.seed(1)
train2 <- sample(1:nrow(Auto_2) , round(nrow(Auto_2)*0.70) )
Auto_2.train <- Auto_2[train2, ]
Auto_2.test <- Auto_2[-train2, ]
```

d) Perform LDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
lda.fit2 <- lda(mpg01 ~ displacement +  horsepower + weight, data=Auto_2,
subset=train2)
lda.fit2

## Call:
## lda(mpg01 ~ displacement + horsepower + weight, data = Auto_2,
##     subset = train2)
##
## Prior probabilities of groups:
##         0         1
## 0.4927007 0.5072993
##
## Group means:
```

```
##   displacement horsepower   weight
## 0     271.9333  129.13333 3611.052
## 1     116.8129   79.27338 2342.165
##
## Coefficients of linear discriminants:
##                         LD1
## displacement -0.0109111922
## horsepower    0.0069843305
## weight       -0.0008795315
```

```
lda.pred2 <- predict(lda.fit2 , Auto_2.test)
table(lda.pred2$class, Auto_2.test$mpg01)
```

```
##
##      0  1
##   0 47  1
##   1 14 56
```

```
mean(lda.pred2$class != Auto_2.test$mpg01)
```

```
## [1] 0.1271186
```

The test error obtained, based on the proportion of incorrect predictions from the confusion matrix, is 12.7%

e) Perform QDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
qda.fit2 <- qda(mpg01 ~ displacement +  horsepower + weight, data=Auto_2,
subset=train2)
qda.fit2
```

```
## Call:
## qda(mpg01 ~ displacement + horsepower + weight, data = Auto_2,
##     subset = train2)
##
## Prior probabilities of groups:
##         0         1
## 0.4927007 0.5072993
##
## Group means:
##   displacement horsepower   weight
## 0     271.9333  129.13333 3611.052
## 1     116.8129   79.27338 2342.165
```

```
qda.pred2 <- predict(qda.fit2 , Auto_2.test)
table(qda.pred2$class, Auto_2.test$mpg01)
```

```
##
##      0  1
```

```
##   0 51  3
##   1 10 54
```

```
mean(qda.pred2$class != Auto_2.test$mpg01)
```

```
## [1] 0.1101695
```

The test error obtained, based on the proportion of incorrect predictions from the confusion matrix, is 11.0% (smaller than the one obtained using LDA).

f) Perform logistic regression on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
log.fit2 <- glm(mpg01 ~  displacement + horsepower + weight,
data=Auto_2.train, family=binomial)
summary(log.fit2)
```

```
##
## Call:
## glm(formula = mpg01 ~ displacement + horsepower + weight, family =
binomial,
##     data = Auto_2.train)
##
## Deviance Residuals:
##     Min       1Q   Median       3Q      Max
## -2.4850  -0.1900   0.1082   0.3534   3.3691
##
## Coefficients:
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)  11.8104207  2.0520434   5.755 8.64e-09 ***
## displacement -0.0136805  0.0062673  -2.183   0.0290 *
## horsepower   -0.0409716  0.0173433  -2.362   0.0182 *
## weight       -0.0019418  0.0008133  -2.387   0.0170 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 379.79  on 273  degrees of freedom
## Residual deviance: 144.51  on 270  degrees of freedom
## AIC: 152.51
##
## Number of Fisher Scoring iterations: 7
```

```
log.probs <- predict(log.fit2 , Auto_2.test, type = "response")
log.pred2 <- rep("0", dim(Auto_2.test)[1])
log.pred2[log.probs > .5] = "1"
table(log.pred2 , Auto_2.test$mpg01)
```

```
##
## log.pred2  0  1
```

```
##         0 53  3
##         1  8 54
```

```
mean(log.pred2 != Auto_2.test$mpg01)
```

```
## [1] 0.09322034
```

The resulting testing error is 0.09% when fitting a logistic regression model.This is lower than the test error obtained with LDA and QDA.

g) Perform naive Bayes on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

```
nb.fit2 <- naiveBayes(mpg01 ~ displacement + horsepower + weight ,
data=Auto_2, subset=train)
nb.fit2
```

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##   0   1
## 0.5 0.5
##
## Conditional probabilities:
##    displacement
## Y       [,1]     [,2]
##   0 273.1582 89.52399
##   1 115.6658 38.42951
##
##    horsepower
## Y        [,1]     [,2]
##   0 130.11224 37.35564
##   1  78.82653 15.91969
##
##    weight
## Y       [,1]     [,2]
##   0 3620.403 676.9322
##   1 2334.765 397.1924
```

```
nb.pred2 <- predict(nb.fit2 , Auto_2.test)
table(nb.pred2, Auto_2.test$mpg01)
```

```
##
## nb.pred2  0  1
##        0 49  2
##        1 12 55
```

```
mean(nb.pred2 != Auto_2.test$mpg01)

## [1] 0.1186441
```

The test error of the naive bayes classifier is 11.9%. This is still a low error, however it is higher than the testing error obtained with logistic regression.

h) Perform KNN on the training data, with several values of K, in order to predict mpg01. Use only the variables that seemed most associated with mpg01 in (b). What test errors do you obtain? Which value of K seems to perform the best on this data set?

- using k=1

```
train2.X <- dplyr::select(Auto_2.train , displacement, horsepower, weight)
test2.X <- dplyr::select(Auto_2.test, displacement , horsepower , weight)
train2.mpg01 <- Auto_2.train$mpg01
set.seed(1)
knn.pred2 <- knn(train2.X, test2.X, train2.mpg01, k=1)
table(knn.pred2, Auto_2.test$mpg01)

##
## knn.pred2  0  1
##         0 51  6
##         1 10 51

mean(knn.pred2 != Auto_2.test$mpg01)

## [1] 0.1355932
```

- using k=10

```
train2.X <- dplyr::select(Auto_2.train , displacement, horsepower, weight)
test2.X <- dplyr::select(Auto_2.test, displacement , horsepower , weight)
train2.mpg01 <- Auto_2.train$mpg01
set.seed(1)
knn.pred2 <- knn(train2.X, test2.X, train2.mpg01, k=10)
table(knn.pred2, Auto_2.test$mpg01)

##
## knn.pred2  0  1
##         0 49  5
##         1 12 52

mean(knn.pred2 != Auto_2.test$mpg01)

## [1] 0.1440678
```

- using k=100

```
train2.X <- dplyr::select(Auto_2.train , displacement, horsepower, weight)
test2.X <- dplyr::select(Auto_2.test, displacement , horsepower , weight)
train2.mpg01 <- Auto_2.train$mpg01
set.seed(1)
```

```
knn.pred2 <- knn(train2.X, test2.X, train2.mpg01, k=100)
table(knn.pred2, Auto_2.test$mpg01)

##
## knn.pred2  0  1
##         0 50  7
##         1 11 50

mean(knn.pred2 != Auto_2.test$mpg01)

## [1] 0.1525424
```

When using KNN with different values of k, we get the lowest testing error from the model with k=1. In this case, the test error is 13.6%, which is higher than all previous models (logistic regression, LDA, QDA, NAive Bayes).

## Chapter 4 - Problem 6

Suppose we collect data for a group of students in a statistics class with variables X1 =hours studied, X2 =undergrad GPA, and Y = receive an A. We fit a logistic regression and produce estimated coefficient, ˆ β0 = −6, ˆ β1 = 0.05, ˆ β2 = 1.

Using the probability form formulation for the logistic regression model, we could get both p(x) and x based on the model parameters. p(X) = (eβ0+β1X1++β2X2) / 1 + eβ0+β1X1+β2X2

a) Estimate the probability that a student who studies for 40 h and has an undergrad GPA of 3.5 gets an A in the class.

```
num <- exp(-6 +0.05*40 + 3.5)
den <- 1 + num
prob <-  num/den
prob

## [1] 0.3775407
```

The probability is 37.8%

b) How many hours would the student in part (a) need to study to have a 50% chance of getting an A in the class?

```
num_hours <- (log(0.5/(1-0.5)) - (-6) - 3.5)/0.05
num_hours

## [1] 50
```

The number of hours needed would be 50 hours.

## Problem 4

Define A, B, C

```
A <- matrix(data= c(1,-0.5,-0.5, 1), nrow=2 , ncol=2)
B <- matrix(data= c(2,-1,-1, 1), nrow=2 , ncol=2)
C <- matrix(data= c(1,-1,-1, 2), nrow=2 , ncol=2)
```

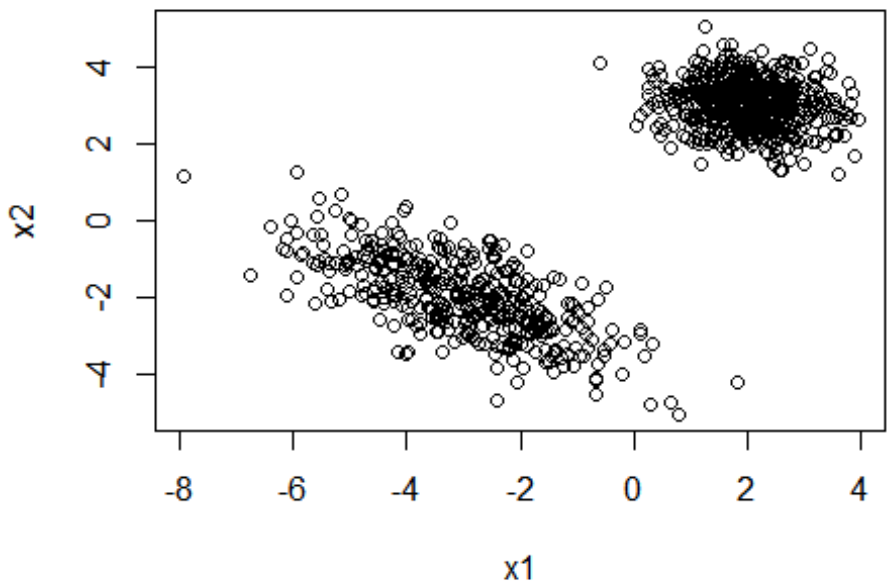
Calculating the variances and covariances matrix for each class (+1: Up, -1: Down), Drawing the samples of size 1000 (with P(Y=1)=0.6) Means for each class are: Class Up = (2,3) , Class Down = (-3,-2) Fit a QDA for each sample

Part a: sample 1

```
Sigma1_up <- solve(A + solve(B))
Sigma1_down <- B
set.seed(1)
sample1_up <- mvrnorm(600, c(2,3), Sigma1_up)
sample1_up <- cbind(sample1_up,rep(1,600))
sample1_down <- mvrnorm(400, c(-3,-2), Sigma1_down)
sample1_down <- cbind(sample1_down,rep(-1,400))
sample1 <- data.frame(rbind(sample1_up, sample1_down))
colnames(sample1) <- c("x1","x2","y")
qda_sample1 <- qda(y ~ x1 + x2, data=sample1)
qda_sample1
```

```
## Call:
## qda(y ~ x1 + x2, data = sample1)
##
## Prior probabilities of groups:
##  -1   1
## 0.4 0.6
##
## Group means:
##           x1        x2
## -1 -3.033698 -1.955800
## 1   2.004473  3.033145
```

```
plot(sample1[,1],sample1[,2], xlab = "x1", ylab = "x2")
```

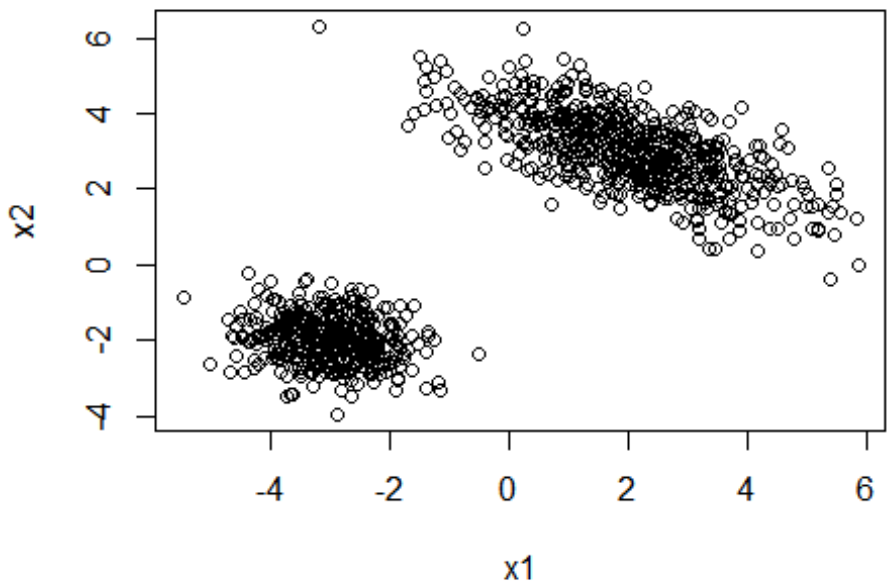



Part b: sample 2

```
Sigma2_up <- B
Sigma2_down <- solve(A + solve(B))
set.seed(1)
sample2_up <- mvrnorm(600, c(2,3), Sigma2_up)
sample2_up <- cbind(sample2_up,rep(1,600))
sample2_down <- mvrnorm(400, c(-3,-2), Sigma2_down)
sample2_down <- cbind(sample2_down,rep(-1,400))
sample2 <- data.frame(rbind(sample2_up, sample2_down))
colnames(sample2) <- c("x1","x2","y")
qda_sample2 <- qda(y ~ x1 + x2, data=sample2)
qda_sample2
```

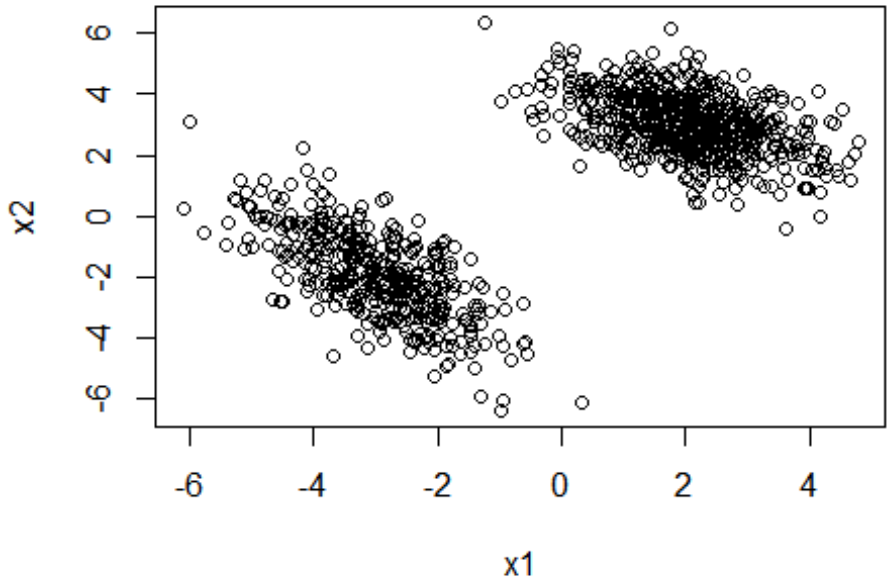
```
## Call:
## qda(y ~ x1 + x2, data = sample2)
##
## Prior probabilities of groups:
##  -1   1
## 0.4 0.6
##
## Group means:
##           x1        x2
## -1 -3.015258 -1.974236
## 1   2.003055  3.040182
```

```
plot(sample2[,1],sample2[,2], xlab = "x1", ylab = "x2")
```




Part c: sample 3

```
Sigma3_up <- A
Sigma3_down <- C
set.seed(1)
sample3_up <- mvrnorm(600, c(2,3), Sigma3_up)
sample3_up <- cbind(sample3_up,rep(1,600))
sample3_down <- mvrnorm(400, c(-3,-2), Sigma3_down)
sample3_down <- cbind(sample3_down,rep(-1,400))
sample3 <- data.frame(rbind(sample3_up, sample3_down))
colnames(sample3) <- c("x1","x2","y")
qda_sample3 <- qda(y ~ x1 + x2, data=sample3)
qda_sample3

## Call:
## qda(y ~ x1 + x2, data = sample3)
##
## Prior probabilities of groups:
##  -1   1
## 0.4 0.6
##
## Group means:
##           x1        x2
## -1 -3.010374 -1.945396
## 1   2.019036  3.038868

plot(sample3[,1],sample3[,2], xlab = "x1", ylab = "x2")
```


x2
6
4
2
0
-2
-4
-6
-6
-4
-2
0
2
4
x1